UNIDADE 4

CRUD. Persistencia de datos e consumo de API REST en *Vue*

Obxectivos da unidade

- **Consumir unha API REST** dende *Vue* empregando ***Axios*** para realizar peticións HTTP (GET, POST, PUT e DELETE).
- Desenvolver un ***CRUD*** completo no *frontend*, xestionando datos de forma estruturada.
- Comprender a **persistencia de datos**, diferenciando entre datos en memoria e datos gardados nun *backend* (*json-server).*
- **Xestionar datos compartidos** entre compoñentes mediante un estado centralizado, mellorando a organización da aplicación.

A.1. Introdución

Na UD4 traballaremos operacións ***CRUD*** reais (crear, ler, actualizar e eliminar datos) dende a aplicación ***Vue***. Para iso necesitamos unha ***API backend*** á que facer peticións *HTTP*.

A idea é **crear unha aplicación para gardar os datos de usuarios e as tarefas asociadas** a cada un deles.

En **aplicacións reais**, esta API adoita estar conectada a bases de datos como *MySQL, PostgreSQL, MongoDB, etc,* pero a configuración dunha delas para traballar con *Vue* suporía ir máis alá do que pretendemos.

Porén, neste curso non imos centrarnos no *backend*, senón en aprender ben:

- Como consumir unha *API* desde *Vue.*
- Como estruturar un *CRUD* no *frontend.*
- Como **xestionar datos compartidos** con estado centralizado.
- Como traballar con peticións *HTTP* reais.

Para acadar estes obxectivos imos empregar **json-server**, ferramenta que nos permite:

- Simular unha *API REST* completa.
- Traballar con datos persistentes.
- Usar **rutas reais** (*GET, POST, PUT, DELETE)..*
- Ver **respostas *HTTP* reais** (200, 404, 500...)

E todo elo sen ter que montar un *backend* complexo.

A clave é esta, e convén insistila varias veces:

- As librarías e a forma de traballar **son as mesmas.**
- ***Axios*** funciona igual con *json-server* que con unha *API* real.

Por exemplo, usar:

- *axios.get('/usuarios')*
- *axios.post('/tarefas', novaTarefa)*
- *axios.put('/usuarios/3', datos)*
- *axios.delete('/tarefas/5')*

Non cambia absolutamente nada se detrás hai:

- *json-server*

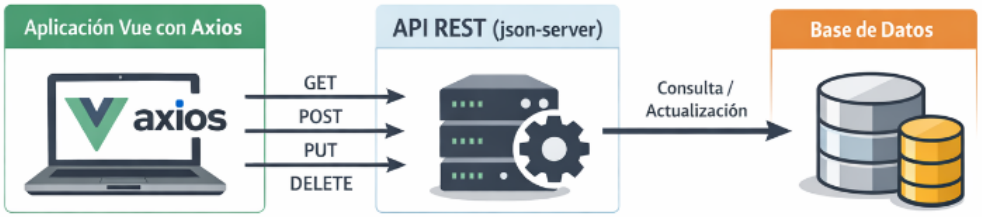
- unha *API* en *Node + Express*
- unha *API* en *Java + Spring*
- unha *API* conectada a *MySQL*

O ***frontend Vue*** non sabe "nin lle importa" que tipo de base de datos hai detrás, de iso encárgase o *backend*; *Vue e Axios* só consumen datos unha API HTTP.

Ademais, cabe sinalar que usaremos como ferramenta ***Axios*** porque é unha das librarías máis empregadas no ecosistema ***Vue*** para facer peticións *HTTP*:

- sinxela de usar,
- moi clara a nivel didáctico
- e ten uso en proxectos profesionais.

Axios actúa como unha **ponte entre a aplicación *Vue* e o *backend***, permitindo comunicarse con calquera *API REST* independentemente da tecnoloxía que haxa detrás.



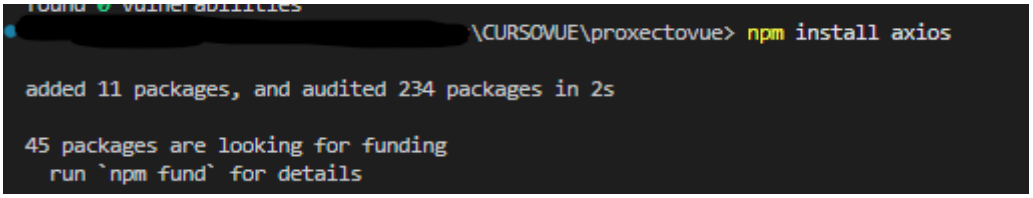
Para finalizar, no **almacén** da Unidade 4 temos os compoñentes:

- *XestiónTarefas.vue*
- *XestiónUsuarios.vue*

Conteñen modificacións con respecto aos anteriores para poder traballar ao longo da unidade de forma cómoda as operacións CRUD. A idea final é dar de alta, baixa, modificar e listar usuarios, así como as tarefas asociadas a cada un en base a un estado centralizado.

Se queredes conservar o anterior, convén crear unha nova rama se estades usando ***GIT*** como repositorio do que estamos a facer.

E xa para entrar en materia e ir adiantando instalamos *axios*:



De vez en cando non vos obviedes de ir actualizando ben con:

> ***npm install*** ou

> ***npm update***

Ao final desta unidade teremos unha pequena aplicación capaz de

- xestionar usuarios e tarefas
- consumir unha API REST real, moi similar ao funcionamento de moitas aplicacións profesionais.

Despois, só e ir engadindo novos compoñentes como pode ser facturas, empregados, produtos... porque a base sempre é a mesma.

A.2. Configuración do *backend* simulado con *json-server*

json-server é unha ferramenta lixeira que nos permite crear unha *API REST,* ficticia pero realista, a partir dun simple ficheiro *JSON*.

Sen configurar servidores reais nin bases de datos complexas, podemos simular un *backend* que nos permite comprobar operacións ***CRUD:***

- *GET*
- *POST*
- *PUT*
- *PATCH*
- *DELETE*

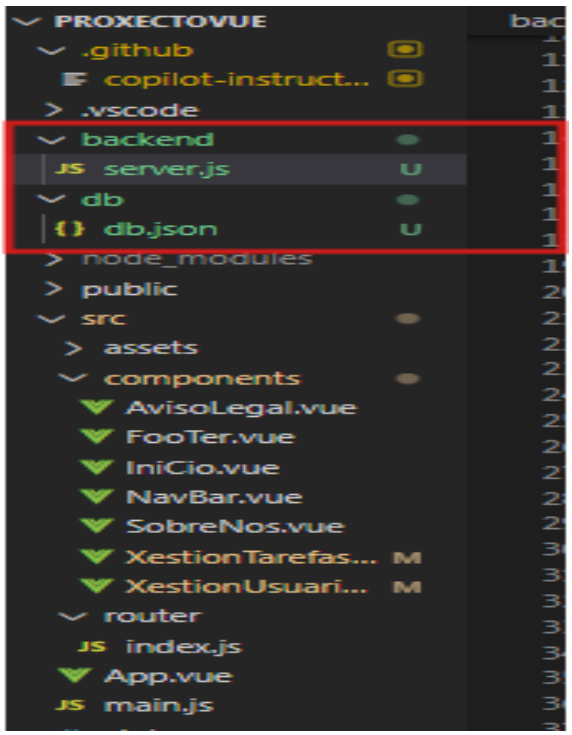
Isto convérteo nunha **ferramenta ideal para aprender, probar e desenvolver aplicacións** *frontend*.

En primeiro lugar imos crear o directorio ***backend*** na raíz do proxecto e, dentro del:

- ***server.js,*** que simulará un *backend* sinxelo con *json-server*

Noutra carpeta que chamamos ***db***, tamén fóra de ***src***:

- ***db.json***, que simulará unha base de datos en formato *JSON*



Por que fóra de *src*?

Porque ***src* está reservado exclusivamente ao *frontend***. ***Vite*** só debe xestionar e compilar código cliente, polo que non debería "ver" nin procesar nada relacionado co *backend*. Ademais, esta estrutura axúdanos a:

- entender mellor a separación entre **cliente e servidor**, un concepto clave cando traballamos con *APIs* e arquitecturas modernas
- evitar problemas de compilación
- e para obter o produto final para produción

Por exemplo:

- en */var/backend* a carpeta cos seus arquivos.
- en */var/www/html* os arquivos compilados do *frontend*.

Continuamos coa instalación:

```
[redacted]\CURSOVUE\proxectovue> npm install json-server@0.17.4 --save-dev

added 65 packages, and audited 223 packages in 3s

43 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

save-dev indica que *json-server* se usa só en desenvolvemento para programar e probar a aplicación, unha aproximación temporal a realidade. En produción, a aplicación conectaría cun servidor real con outro tipo de servidor de base de datos.

Nota.- *Por cuestións de compatibilidade con algúns prototipos convén empregar a versión indicada nos apuntes e non a máis recente (superior á 0.18). Probablemente estes problemas estarán solucionados en futuras versións ou no momento de realizar este curso.*

```
[redacted]\CURSOVUE\proxectovue> npm install concurrently --save-dev

added 25 packages, and audited 108 packages in 3s

27 packages are looking for funding
  run `npm fund` for details
```

Por outra banda imos instalar unha ferramenta ***concurrently*** que nos permite arrincar os dous servidores *dev* e *json-server* nunha soa tirada.

Imos a ***package.json*** e retocamos:

"scripts": {

"start": "concurrently \"vite\" \"json-server—watch db/db.json—port 3000\""

}

```
{
  "name": "proxectovue",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  ▷Debug
  "scripts": {
    "dev": "vite",
    "start": "concurrently \"vite\" \"json-server --watch db/db.json --port 3000\"",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "vue": "^3.5.18",
```

De esta forma executando ***npm run start*** arrincamos os dous servidores.

De esta forma executando ***npm run start*** arrincamos os dous servidores cun só comando.

- O servidor de Vite → *http://localhost:5173*
- O servidor *json-server* → *http://localhost:3000*

```
[redacted]-\CURSOVUE\proxectovue> npm run start

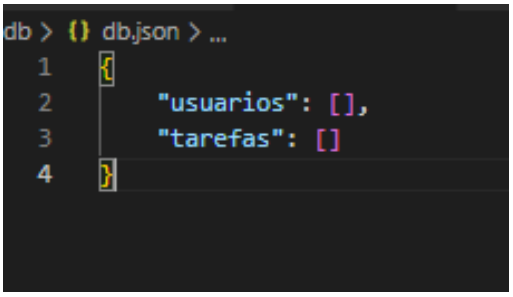
> proxectovue@0.0.0 start
> concurrently "vite" "nodemon backend/server.js --watch backend/server.js"

[1] [nodemon] 3.1.11
[1] [nodemon] to restart at any time, enter `rs`
[1] [nodemon] watching path(s): backend\server.js
[1] [nodemon] watching extensions: js,mjs,cjs,json
[1] [nodemon] starting `node backend/server.js`
[0]
[0]   VITE v7.3.0  ready in 429 ms
[0]
[0]   ➜  Local:   http://localhost:5173/
[0]   ➜  Network: use --host to expose
[1] Servidor iniciado en http://localhost:3000
```

Neste caso non executamos *json-server* directamente desde o *script*, senón a través o ficheiro ***server.js*** (ver código comentado). Isto permítenos configurar mellor o servidor e definir a ruta ao ficheiro db.json dentro do propio código.

Por certo, para **parar** os servidores***: Ctrl+C***

O arquivo ***db.json*** ten que estar xa preparado:



Con isto temos configurado xa noso ***backend*** simulado.

Prestade moita atención de que estea ben escrito, xa que se falta algunha coma, chaves ou o que sexa fallarán as operacións *CRUD* xa que *json-server* non funcionará se o ficheiro JSON non é válido.

Se tes dúbidas co formato, podes validar o JSON en ferramentas online ou no propio editor (VS Code marca os erros automaticamente).

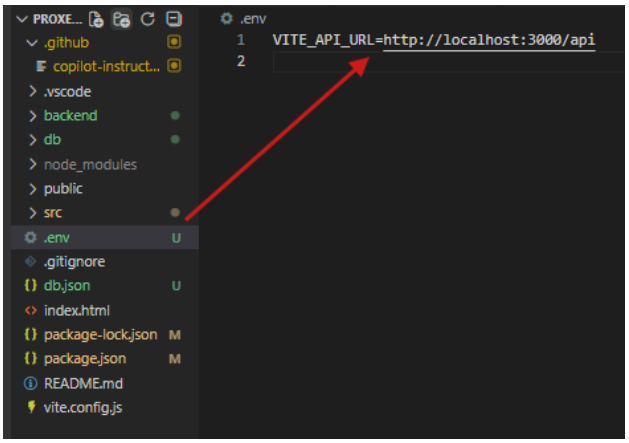
E con isto xa temos configurado o noso ***backend*** simulado.

Pero imos ir un paso máis alá.

A.3. Operacións CRUD con *json-server*

Primeiro facemos *CRUD* cos usuarios porque son a **entidade principal da aplicación.** Unha vez dominado neste compoñente, as operacións *CRUD* das **tarefas** serán iguais e so temos que engadir outro recurso relacionado a través dun estado xeral para relacionar os usuarios coas súas tarefas.

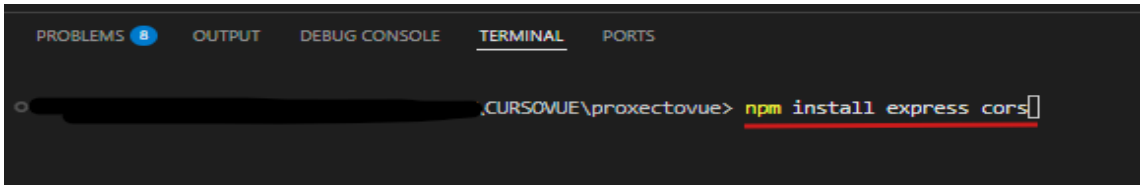
Empezamos creamos un arquivo ***.env*** para albergar a ruta de xestión das operacións *CRUD* e centralizar a configuración da aplicación, evitando ter valores fixos no código e facilitando o cambio da *URL* da *API* entre contornas de desenvolvemento e produción.



Paso 1. Configuración do *server.js*

Temos o arquivo ***server.js*****,** moi básico, que o punto de inicio da nosa API simulada.

Temos que instalar ***express*** e ***cors*** para crear un *backend* o máis próximo a realidade.



Express é un *framework* lixeiro e flexible para ***Node.js*** que nos permite crear servidores **web** e *APIs* de forma rápida e estruturada. Facilita tarefas como:

- a xestión de rutas
- xestión de peticións e repostas *HTTP*
- uso de *middlewares* e integración con bases de datos, evitando ter que escribir todo o código de baixo nivel de *Node* puro

- é a base de moitas ***APIs REST*** e proxectos *backend* en ***JavaScript***, permitindo centralizar a lóxica do servidor e ofrecer datos ao *frontend* de forma organizada.

No noso caso, empregámolo como contedor do servidor, achegando unha estrutura máis realista ao proxecto. que servirá de ponte entre ***json-server*** e a nosa aplicación *Vue.*

***CORS* (*Cross-Origin Resource Sharing*)** é un **mecanismo de seguridade** que permite ou restrinxe que un *frontend* que se executa nun **dominio ou porto.**

Por exemplo:

- *http://localhost:5173* *Frontend*
- *http://localhost:3000* *Backend*

O *middleware* ***cors*** para *Express* xestiona estas regras e habilita o acceso seguro desde o *frontend* ao *backend* cando se están a desenvolver aplicacións en arquitecturas separadas.

Falemos do arquivo ***server.js.*** A función do ***server.js*** é:

- inicializar o servidor (***Express + json-server***)
- aplicar configuración globais como ***CORS***
- integrar ***json-server*** como API simulada
- ser o punto único de arranque da aplicación

O fluxo queda organizado do seguinte xeito:

- ***server.js:*** arranque do servidor
- ***json-server:*** xera a *API REST* automaticamente
- ***db.json:*** base de datos simulada
- ***Vue*:** consume os datos mediante *Axios*

A configuración do ***server.js*** dentro do *backend* será o que se mostra na imaxe seguinte.

ome > jcarlos > Descargas > JS server.js > ...

```
import express from "express";          // Framework para crear o servidor
import cors from "cors";                // Permite comunicación entre frontend e
import jsonServer from "json-server";   // Simula unha API REST a partir dun JSON
import path from "path";                // Xestión de rutas de ficheiros
import { fileURLToPath } from "url";    // Necesario para ES Modules

const app = express();                  // Creamos a aplicación Express
const PORT = 3000;                      // Porto onde se executa o servidor

// ------------------------------------------------
// Adaptación para ES Modules (non existe __dirname)
// ------------------------------------------------
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// ------------------------------------------------
// Middleware básico
// ------------------------------------------------
app.use(cors());
// Permite que o frontend (por exemplo Vite en localhost:5173)
// poida facer peticións ao backend (localhost:3000)

// ------------------------------------------------
// json-server: crea a API a partir de db.json
// ------------------------------------------------
const router = jsonServer.router(
  path.join(__dirname, "../db/db.json")
);
// Converte o ficheiro db.json nunha API REST completa
// (GET, POST, PUT, DELETE)

// Middlewares propios de json-server (logger, estáticos, etc.)
const middlewares = jsonServer.defaults();

app.use(middlewares);
// Aplica os middlewares internos de json-server

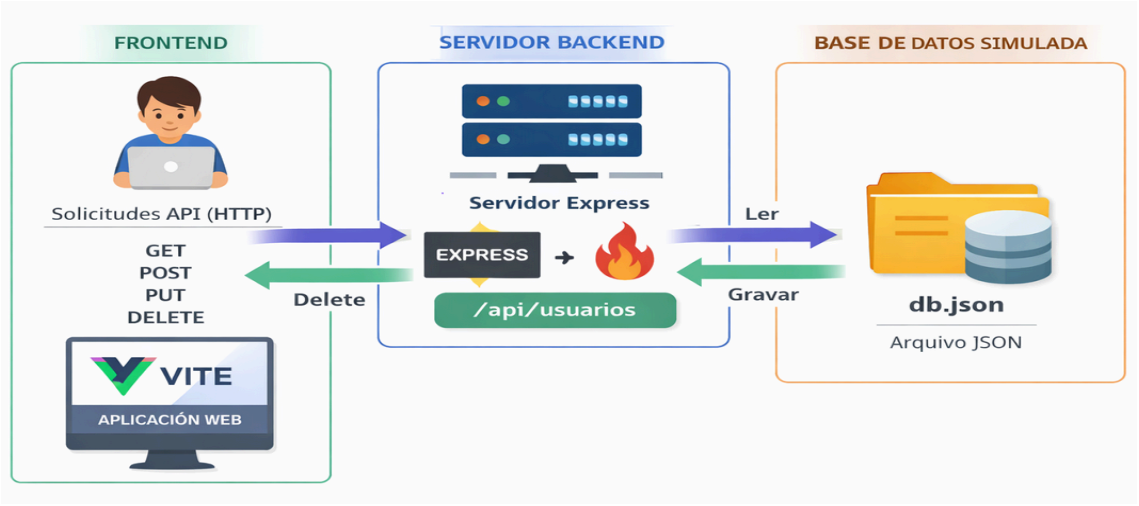
// ------------------------------------------------
// Montaxe da API
// ------------------------------------------------
app.use(router);
// Expón directamente as rutas:
// /usuarios
// /tarefas
// sen prefixos adicionais para evitar conflitos

// ------------------------------------------------
// Arranque do servidor
// ------------------------------------------------
app.listen(PORT, () => {
  console.log(`API simulada en http://localhost:${PORT}`);
});
// Inicia o servidor no porto indicado
```

Importante: este **arquivo está no almacén da UD4.** Pero as modificacións posteriores que fagamos xa corren da túa conta.

Na Rede (e os propios axentes IA para programación) poden propoñerte outras configuracións de *server.js,* pero o importante é entendelo seu funcionamento.

O **esquema de traballo** resúmese na imaxe seguinte.



Antes de continuar é necesario aclarar unha cousa: ***Express* non é estritamente necesario.**

***json-server* permite crear unha API REST completa sen necesidade dun *backend* tradicional.**

Coa simple existencia dun ficheiro *db.json,* é capaz de:

- Levantar un servidor *HTTP*
- Crear automaticamente unha *API REST*
- Ler datos desde o ficheiro *db.json*
- Escribir datos no *db.json* (persistencia simulada)
- Xestionar operacións *CRUD* completas:
    - *GET*: lectura
    - *POST:* creación
    - *PUT / PATCH:* actualización
    - *DELETE*: eliminación

Pero nós engadimos *Express* por **motivos didácticos** e **estruturais**, **non por necesidade técnica** e así poder acercarnos o máximo a un sistema dos que habitualmente se usan.

Nunha **aplicación profesional** normalmente temos**:**

- un servidor (Express, *Spring, Laravel*...)
- unha API
- unha base de datos

json-server simplifica demasiado este esquema**,** así que ***Express*** **axuda a que a estrutura sexa máis realista.**

Express permítenos ter un único ficheiro *server.js* que:

- inicializa a aplicación
- configura *middlewares* (como *CORS*)
- integra *json-server*

E no futuro xa nos queda a aplicación preparada para elementos *que json-server* non pode por si só**:**

- unha base de datos real
- *authenticación*
- validación
- *middlewares* adicionais

Podemos mellorar agora o arranque de ***server.js***.

Ata agora temos que detelo ***(Ctrl + C)*** e arrincalo manualmente cada vez que fagamos un cambio, pero iso resulta pouco eficiente xa que no período de desenvolvemento as paradas e arranques son habituais polo que se usan ferramentas como ***nodemon***, que cada vez que detecta os cambios no código e **reinicia automaticamente o servidor** cando detecta modificacións

Podemos ir un paso máis alá, e integralo en *concurrently* no arquivo *package.json*, para que inclúa ***nodemon*** no *backend*, de modo que cada vez que cambiemos o código do servidor, este **se recargue automaticamente** sen ter que reinicialo manualmente, facendo o desenvolvemento máis áxil executando só unha vez ***npm run start***

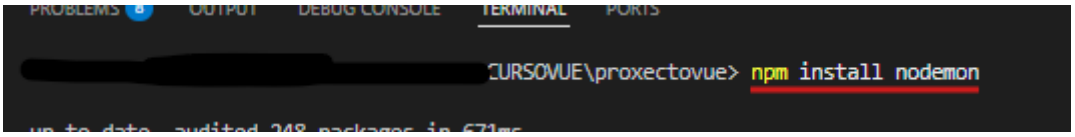
```
"scripts": {
  ...
```

"start": "concurrently \"vite\" \"nodemon backend/server.js—watch backend/server.js\"",

....

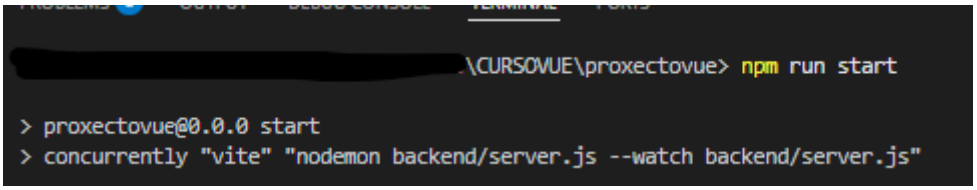
},

Non esquecemos de instalar ***nodemon.***





Arrincamos e xa temos o servidor funcionando.



Imos ao *frontend.*

Paso 3. Quédanos conectar a zona de ***script*** de *XestionUsuarios.vue* co *backend*.

Imos a ir, un por un, para que vaias modificando o que temos no zona de *script* e ir probando.

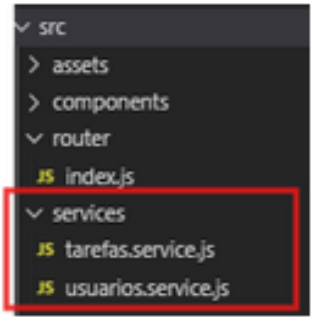
Antes de comezar coas operacións CRUD, imos preparar a capa de comunicación coa API.

Para iso, creamos unha nova carpeta no proxecto:

src/services

Dentro dela creamos o ficheiro:

usuarios.service.js
tarefas.service.js



Nas unidades anteriores traballabamos con datos gardados na memoria do navegador, polo que as operacións eran inmediatas e non dependían de ningún servidor.

Agora, ao traballar cunha *API*, as peticións poden tardar en responder, xa que dependen dunha comunicación *HTTP.*

Por outra banda, empregamos funcións asíncronas ***async*** porque:

- A resposta do servidor non é inmediata
- Pode haber retrasos na rede
- Non queremos bloquear a aplicación

Ao marcar unha función como ***async***, indicamos que pode executarse de forma non bloquéante.

A palabra clave ***await*** permítenos:

- Esperar a resposta dunha petición
- Continuar a execución cando esa resposta chega
- Manter o código claro e lexible

Tampouco debemos esquecernos das seguintes importacións que son:

- ***Axios*** a librería que se comunica con *backend*
- ***ref*** e ***reactive,*** dependendo da complexidade da estrutura de datos
- ***API_URL*** recorda que usamos ***.env*** para manexar as ***URL***

```
<script setup>
import { ref, reactive } from "vue";
import axios from "axios";
import { onMounted } from "vue";

//URL dende .env
const API_URL = import.meta.env.VITE_API_URL;

const usuarios = ref([]); // Array que se conectará co json-server

const novoUsuario = reactive({
  dni: "",
  nome: "",
  correo: "",
  provincia: "",
  activo: false,
  tipoCuenta: "",
});

const usuarioSeleccionado = ref(null);
const tarefasUsuario = ref([]);
```

- **GET – obter usuarios**

Comezamos coa **primeira operación *CRUD***, que é obter o listado de usuarios da base de datos ***GET.*** Ao principio vemos que o nome está en cor gris, iso significa que aínda non se está a usar.

Empezamos con arquivo ***usuarios.service.js*** e a operación CRUD

```
services > JS usuarios.service.js > ...
  import axios from "axios";

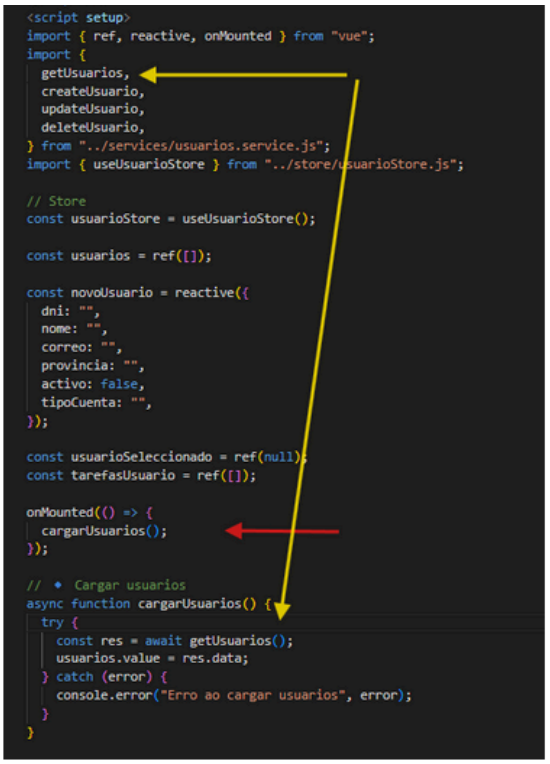
  const API_URL = import.meta.env.VITE_API_URL;

  // GET usuarios
  export const getUsuarios = async () => {
    return await axios.get(`${API_URL}/usuarios`);
  };
```

Tamén é válida esta configuración.

```
export const getUsuarios = async () => {
        const res = await axios.get(`${API_URL}/usuarios`);
        return res.data;
};
```

Sen embargo, nunha aplicación deste estilo nos interesa que nos mostre os usuarios da base de datos **cada vez que carguemos o compoñente.** Recorda que isto faciámolo co *hook* ***onMounted*().** Pois entón só temos en engadir o seguinte:



```
<script setup>
import { ref, reactive, onMounted } from "vue";
import {
  getUsuarios,
  createUsuario,
  updateUsuario,
  deleteUsuario,
} from "../services/usuarios.service.js";
import { useUsuarioStore } from "../store/usuarioStore.js";

// Store
const usuarioStore = useUsuarioStore();

const usuarios = ref([]);

const novoUsuario = reactive({
  dni: "",
  nome: "",
  correo: "",
  provincia: "",
  activo: false,
  tipoCuenta: "",
});

const usuarioSeleccionado = ref(null);
const tarefasUsuario = ref([]);

onMounted(() => {
  cargarUsuarios();
});

// 🔹 Cargar usuarios
async function cargarUsuarios() {
  try {
    const res = await getUsuarios();
    usuarios.value = res.data;
  } catch (error) {
    console.error("Erro ao cargar usuarios", error);
  }
}
```

Observamos que o compoñente ***XestiónUsuarios.vue*** importa a función ***getUsuarios*** desde o ficheiro **usuarios.service.js.**

Esta función é a encargada de realizar a petición HTTP á API para obter os datos dos usuarios.

Posteriormente, o compoñente chama a esta función e utiliza o resultado para cargar os datos na interface na zona do *template*.

- **POST – engadir usuarios**

Imos agora coa segunda operación CRUD, **POST**, que permite engadir novos usuarios á base de datos.

Deste xeito, os usuarios creados poderán mostrarse automaticamente na táboa.

Se observamos o código, vemos que o estamos a reutilizar, vemos que a función *gardarUsuario* realiza dúas operacións segundo o caso:

Se non hai usuario seleccionado: crea un novo usuario (POST)

Se hai usuario seleccionado; actualiza os datos (PUT)

É dicir, utilizamos o **mesmo botón para crear e modificar usuarios.**

Arquivo *usuarios.service.js*

```
// POST usuario
export const createUsuario = async (usuario) => {
  return await axios.post(`${API_URL}/usuarios`, usuario);
};
```

Ou tamén,

```
export const createUsuario = async (usuario) => {
    const res = await axios.post(`${API_URL}/usuarios`, usuario);
    return res.data;
};
```

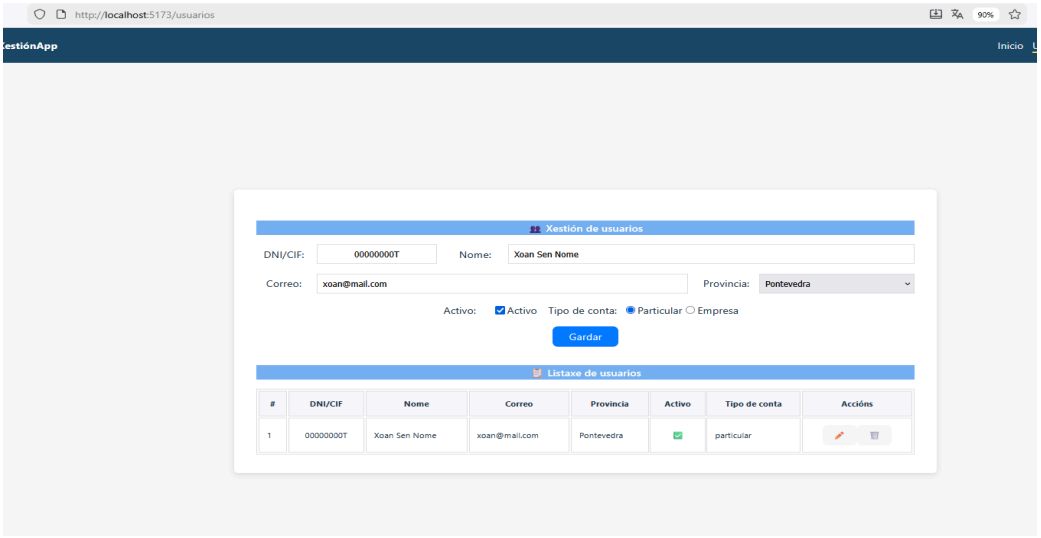
E a zona de *script* do arquivo *Vue*.

```
// ✦ Crear ou actualizar
async function gardarUsuario() {
  try {
    if (usuarioSeleccionado.value) {
      await updateUsuario(usuarioSeleccionado.value.id, {
        ...novoUsuario,
      });
    } else {
      await createUsuario(novoUsuario);
    }

    usuarioStore.seleccionarUsuario(
      usuarioSeleccionado.value || novoUsuario
    );

    await cargarUsuarios();
    limparFormulario();
  } catch (error) {
    console.error("Erro ao gardar usuario", error);
  }
}
```

Abrimos a aplicación e introducimos uns datos:





Se observamos o código, cando gardamos un usuario (*POST*) carga a táboa chamando a *cargarUsuarios* e limpa o formulario (opcional) chamando a función *limparFormulario()* que como son datos da memoria do navegador non é necesario *await.*

```
// Limpar formulario
function limparFormulario() {
  novoUsuario.dni = "";
  novoUsuario.nome = "";
  novoUsuario.correo = "";
  novoUsuario.provincia = "";
  novoUsuario.activo = false;
  novoUsuario.tipoCuenta = "";
  usuarioSeleccionado.value = null;
  tarefasUsuario.value = [];
}
```

- **PUT – modificar usuarios**

Por outra banda, o mesmo botón de **Gardar** tamén se utiliza para modificar os datos dun usuario existente mediante a operación **PUT**.

Isto é posible porque ***json-server* identifica cada usuario a través dun *id* único**.

Para poder editar un usuario, o botón coa icona do lapis permite:

- Seleccionar o usuario a editar
- Cargar os seus datos no formulario

Deste xeito, o formulario pasa a modo edición.

Arquivo ***usuarios.service.js***

```
// PUT usuario
export const updateUsuario = async (id, usuario) => {
  return await axios.put(`${API_URL}/usuarios/${id}`, usuario);
};
```

Ou tamén,

```
export const updateUsuario = async (id, usuario) => {
    const res = await axios.put(`${API_URL}/usuarios/${id}`, usuario);
   return res.data;
};
```

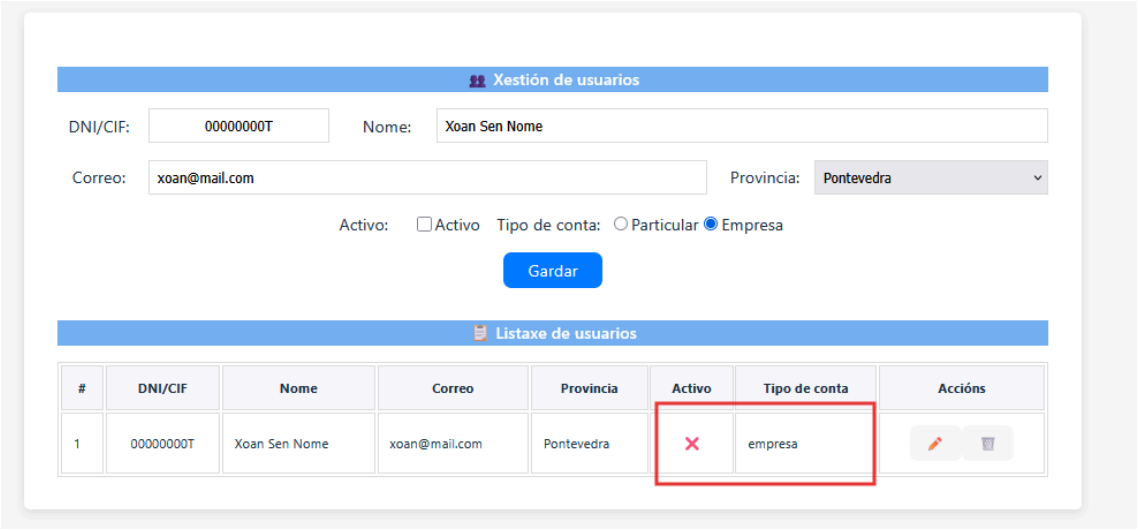
E a zona de *script* do arquivo *Vue* que é a mesma función de *gardarUsuario*

```
// ♦ Crear ou actualizar
async function gardarUsuario() {
  try {
    if (usuarioSeleccionado.value) {
      await updateUsuario(usuarioSeleccionado.value.id, {
        ...novoUsuario,
      });
    } else {
      await createUsuario(novoUsuario);
    }

    usuarioStore.seleccionarUsuario(
      usuarioSeleccionado.value || novoUsuario
    );

    await cargarUsuarios();
    limparFormulario();
  } catch (error) {
    console.error("Erro ao gardar usuario", error);
  }
}
```

Temos a imaxe onde cargamos con botón do lapis un usuario.



Cando se confirma o gardado:

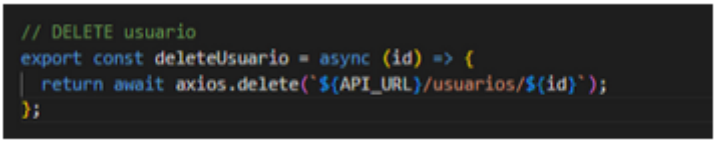
- Se existe un usuario seleccionado → realízase un **PUT**
- Os datos actualízanse na API
- A lista de usuarios recárgase automaticamente

- **DELETE – eliminar usuarios**

A última operación CRUD é a eliminación de usuarios mediante a operación **DELETE**.
Para iso, utilizamos o botón representado coa icona dun cubo de lixo.
Cada usuario está identificado de forma única por un id xerado por *json-server.*
Cando premermos no botón de eliminar:

- Obtemos o *id* do usuario seleccionado
- Enviamos unha petición DELETE á *API*
- *json-server* elimina ese rexistro do *db.json*
- Recárgase a lista de usuarios
- A táboa actualízase automaticamente
- usuario desaparece da interface

O arquivo *usuarios.service.js*



```
// DELETE usuario
export const deleteUsuario = async (id) => {
  return await axios.delete(`${API_URL}/usuarios/${id}`);
};
```

Ou tamén,

```
export const deleteUsuario = async (id) => {
        const res = await axios.delete(`${API_URL}/usuarios/${id}`);
        return res.data;
};
```

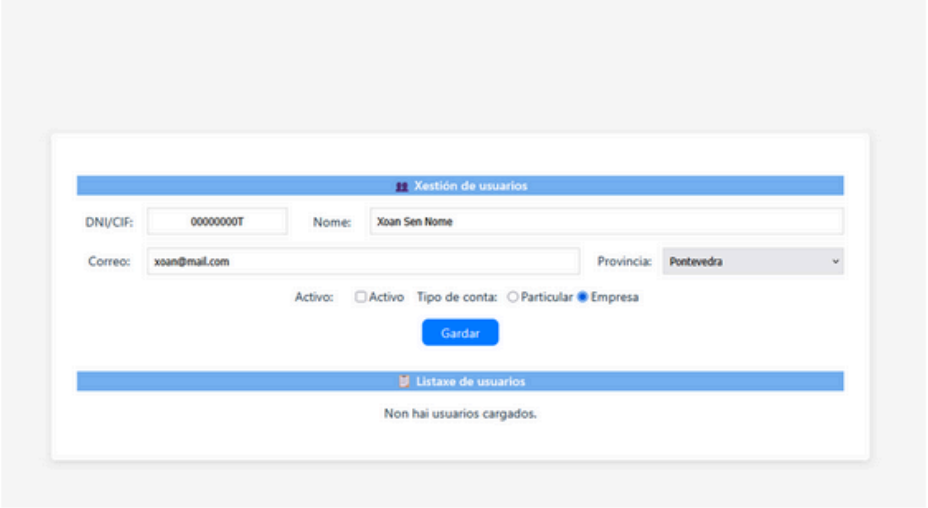
E a zona de *script* do arquivo *Vue*

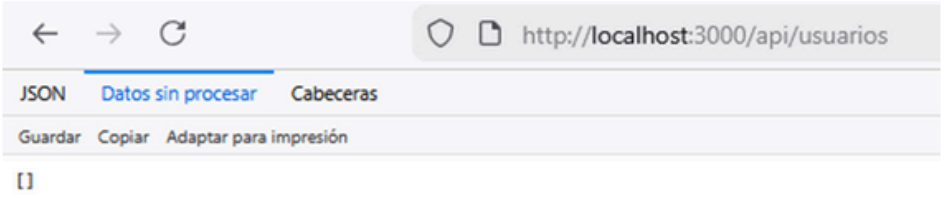
```
// 🔹 Eliminar
async function eliminarUsuario(index) {
  const usuario = usuarios.value[index];

  try {
    await deleteUsuario(usuario.id);

    await cargarUsuarios();

    if (
      usuarioSeleccionado.value &&
      usuarioSeleccionado.value.id === usuario.id
    ) {
      limparFormulario();
      usuarioStore.limparUsuario();
    }
  } catch (error) {
    console.error("Erro ao eliminar usuario", error);
  }
}
```





Agora queda *tarefas.service.js*. É unha vez feito facer as probas pertinentes. Tede en conta que algunhas destas función se modificarán e outras novas aparecerán.

src > services > JS tarefas.service.js > ...

```
import axios from "axios";

const API_URL = import.meta.env.VITE_API_URL;

// listar GET
export const getTarefas = async () => {
  return await axios.get(`${API_URL}/tarefas`);
};

// alta POST
export const createTarefa = async (tarefa) => {
  return await axios.post(`${API_URL}/tarefas`, tarefa);
};

// update PUT
export const updateTarefa = async (id, tarefa) => {
  return await axios.patch(`${API_URL}/tarefas/${id}`, tarefa);
};
```

Hai un punto que apenas nos fixamos que é o ***template,*** o cal ten un par de cousas interesantes especialmente na zona da táboa.

Imos comentala pero antes mostramos o código.

```
</thead>
<tbody>
  <tr v-for="(u, index) in usuarios" :key="index">
    <td>{{ index + 1 }}</td>
    <td style="text-align: center">{{ u.dni }}</td>
    <td>{{ u.nome }}</td>
    <td>{{ u.correo }}</td>
    <td>{{ u.provincia }}</td>
    <td style="text-align: center">{{ u.activo ? "✅" : "❌" }}</td>
    <td>{{ u.tipoCuenta }}</td>
    <td style="text-align: center">
      <button @click="editarUsuario(index)" title="Editar">🖍️</button>
      <button @click="eliminarUsuario(index)" title="Eliminar">🗑️</button>
    </td>
  </tr>
```

nos fixamos na expresión:

{{ u.activo ? "✅" : "❌" }}

Se nos fixamos en ***u.activo*** actúa como una directiva ***v-if,*** aínda que en realidade trátase de un **operador ternario.**

O seu funcionamento é:

- Se *u.activo* é *true*: móstrase ✅
- Se *u.activo* é *false*: móstrase ❌

Este tipo de expresións permiten facer condicións simples directamente no template de forma moi clara e concisa.

```
    </div>
    <button
      type="submit"
      class="btn-guardar"
      :disabled="novoUsuario.dni === '' || novoUsuario.nome === ''"
    >
      Gardar
    </button>
```

No botón de gardar atopamos outro punto interesante:

:disabled="novoUsuario.dni === '' || novoUsuario.nome === ''"

Aquí utilizamos ***v-bind*** (abreviado como **:**) para enlazar dinámicamente o atributo ***disable***.

O botón:

- **desactívase** se o DNI ou o nome están baleiros
- **actívase** cando ambos teñen valor

Estes exemplos amosa moi ben unha vez máis as vantaxes da reactividade en *Vue*:

- O ***template*** encárgase da interface
- O ***script*** encárgase da lóxica

Esta separación permite:

- código máis limpo
- mellor organización
- maior facilidade de mantemento

Ademais, neste caso, o estado dos datos controla o comportamento da interface **con renderizados condicionais.**

Evitamos que o usuario poida enviar:

- formularios incompletos
- datos inválidos

Isto é un exemplo claro de **reactividade en *Vue***, onde a interface responde automaticamente aos cambios nos datos.

A.4. Estado global e comunicación entre vistas con Pinia

Ata o de agora estivemos traballando cos datos de forma local dentro de cada compoñente.

Isto funciona ben mentres toda a lóxica está nun único compoñente, pero comeza a quedar limitado cando necesitamos **compartir información entre vistas distintas e o proxecto escala en complexidade.**

No noso caso, ao seleccionar un usuario en *XestiónUsuarios.vue* precisamos **gardar o seu *id* e o seu *nome*** para poder xestionar as tarefas asociadas a ese usuario en *XestiónTarefas.vue*, que se carga noutra vista diferente.

Como estes compoñentes **non teñen relación pai-fillo directa,** que é o habitual en proxectos complexos, non é práctico nin recomendable usar ***props*** nin ***emits***. En compoñentes onde esa **xerarquía non está clara** ou simplemente **non existe**, **convén usar estados centralizados.**

Un **estado centralizado** actuará como **unha zona común** onde gardar os datos do usuario seleccionado, de forma que poidan ser utilizados por distintos compoñentes da aplicación. Deste xeito:

- *XestiónUsuarios.vue* poderá actualizar o usuario seleccionado.
- *XestiónTarefas.vue* poderá **ler ese mesmo usuario** en calquera momento.
- Evitamos duplicar datos e mantemos a aplicación máis organizada e escalable.

Este enfoque é o que se emprega habitualmente **en aplicacións reais cando diferentes partes da aplicación necesitan acceder á mesma información**, independentemente da vista ou compoñente no que se atopen cando nos movemos entre os compoñentes.

Para estas situacións imos empregar ***Pinia***, o sistema oficial de xestión de estado de *Vue*.

Pinia permítenos definir un **estado global**, accesible desde calquera compoñente da aplicación, independentemente da vista na que se atope.

Esta ferramenta substitúe a outras solucións máis sinxelas como ***provide*** e ***inject***, ambas ferramentas permiten compartir datos entre compoñentes con xerarquía sen usar ***props***, pero **non está pensado para compartir estado entre vistas, pero sobre todo para estados complexos**.

Por este motivo, en aplicacións reais empréganse solucións de estado global como ***Pinia*** que é a ferramenta máis usada neste ámbito

Empecemos con **Pinia**.

Paso 1. Instalación

```
[redacted]\CURSOVUE\proxectovue> npm install pinia

added 13 packages, and audited 247 packages in 3s

49 packages are looking for funding
```

Paso 2. Rexistro de *Pinia* na aplicación

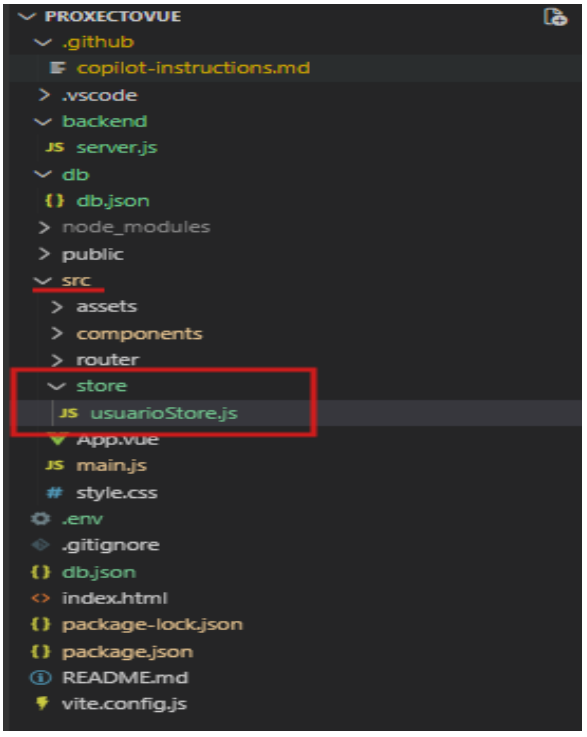
Unha vez instalada, debemos rexistrar *Pinia* na aplicación principal. Para iso modificamos o arquivo ***main.js***

```
src > JS main.js > ...
1   import { createApp } from 'vue'
2   import { createPinia } from 'pinia'
3   import './style.css'
4   import App from './App.vue'
5   import router from './router'
6
7   const pinia = createPinia()
8
9   createApp(App).use(router).use(pinia).mount('#app')
```

Paso 3. Estrutura básica do estado global

Un ***store*** é simplemente un **módulo onde centralizamos datos e funcións** relacionadas con eses datos.

Creamos o ***store*** onde almacenaremos o **id** e o **nome.** Hai que ter en conta que interésanos almacenar os datos só ao ***seleccionUsuario*** e tamén para ***limparUsuario*** que é o noso contexto de traballo.



Pódese pensar que crear o **store** en ***src*** pode supor un problema de seguridade dependendo da sensibilidade dos datos: *nome, id.*

Outra cousa sería gardar *tokens, claves API* ou *contrasinais* que non é o caso e iso se validaría e/ou **gardaríase no *backend*.**

En resumo, *Pinia* é un almacén de estado do *frontend*, non un sistema de seguridade que nunca debe almacenar datos sensibles.

No noso caso, imos gardar o **usuario seleccionado** nun estado global con ***Pinia***. Así, cando neste caso se selecciona na listaxe un usuario, gardaremos o seu *id* e nome no ***store***. De esta forma, calquera **outro compoñente ou vista poderá acceder a estes datos** sen necesidade de *props* ou eventos, mantendo a aplicación organizada e facilitando a xestión de tarefas asociadas a cada usuario, e dicir

1. Seleccionas un usuario en *XestionUsuarios*
2. Garda os datos no ***usuarioStore.js***
3. En *XestionTarefas* colle os datos do *store*

1. Empecemos coa configuración de ***usuariosStore.js.*** O código xa está no almacén comentado pero imos recordalo. A primeira parte consiste na importación do módulo *Pinia* e na definición dunha variable ***Store*** que conteña os dous datos que nos interesan, o ***id*** e o ***nome*** do usuario do cal imos xestionar a súas tarefas.
   Despois temos dúas funcións: a acción que toman os valores en cuestión, ***seleccionarUsuario*** e a que borran eses valores ***limparUsuario,*** cando abandonamos o compoñente ***XestionTarefas.vue.***
   En resumo, definimos o estado ***(id e nome***) e as accións asociadas.

src > store > JS usuarioStore.js > ...

```
1  import { defineStore } from "pinia";
2
3  // Definimos a store para xestionar o usuario seleccionado
4  export const useUsuarioStore = defineStore("usuario", {
5  // Estado inicial da store
6    state: () => ({
7      id: null,
8      nome: null,
9    }),
10
11   // Acciones para modificar o estado da store
12   actions: {
13     seleccionarUsuario(usuario) {
14       this.id = usuario.id;
15       this.nome = usuario.nome;
16     },
17
18     limparUsuario() {
19       this.id = null;
20       this.nome = null;
21     },
22   },
23 });
24
```

2. Engadimos os seguintes códigos en *XestiónUsuarios.vue,* o que nos permiten **facer a chamada ao store para gardar o usuarios seleccionado.** Basicamente estará na función ***gardarUsuario*** e na función ***editarUsuario*** que son as dúas que poden requirir que se garden os datos do usuario para usalos en outros compoñentes, neste caso en *XestionTarefas.vue.* Non esquecer que hai importar tamén o ***Store.***
   Para facilitar a lectura tomamos **só as zonas de código que se engaden**.
   As modificación nas zonas de importación e declaracións:



   O que engadimos en *gardarUsuario()*

```
// CREAR OU MODIFICAR USUARIO

async function gardarUsuario() {
  try {
    if (usuarioSeleccionado.value) {
      await axios.put(
        `${API_URL}/usuarios/${usuarioSeleccionado.value.id}`,
        { ...novoUsuario }
      );
    } else {
      await axios.post(`${API_URL}/usuarios`, novoUsuario);
    }

    // Actualizamos no Store o estado do usuario seleccionado store ou o novo usuario
    usuarioStore.seleccionarUsuario(usuarioSeleccionado.value || novoUsuario);

    await cargarUsuarios();
    limparFormulario();
  } catch (error) {
    console.error("Erro ao gardar usuario", error);
  }
}
```

   O código engadido en *editarUsuario.*

```
function editarUsuario(index) {
  const usuario = usuarios.value[index];
  Object.assign(novoUsuario, usuario);   // Copiamos os datos do usuario ao formulario
  usuarioSeleccionado.value = usuario;  // Marcamos o usuario como seleccionado para editar
  usuarioStore.seleccionarUsuario(usuario); // Actualizamos o Store co usuario seleccionado

  tarefasUsuario.value = []; // Sen tarefas de exemplo
}
```

E cando elimínamos o usuario, eliminamos o ***Store***

```
function eliminarUsuario(index) {
  const usuario = usuarios.value[index];
  axios.delete(
    `${API_URL}/usuarios/${usuario.id}`)
    .then(() => {
      cargarUsuarios();
      // Se o usuario eliminado era o seleccionado, limpamos o formulario e o Store
      if (usuarioSeleccionado.value && usuarioSeleccionado.value.id === usuario.id) {
        limparFormulario();
        usuarioStore.limparUsuario();
      }
    })
    .catch((error) => {
      console.error("Erro ao eliminar usuario", error);
    });
}
```

3. Quédanos as modificacións en ***XestiónTarefas.vue.*** Este é o compoñente que fai uso dos datos gardados no *Store.*

   Empeza coa importación e declaración da variable.

```
<script setup>
import { ref, onMounted } from 'vue'
import { useUsuarioStore } from '../store/usuarioStore.js'

// Estes datos virán do store ou das peticións HTTP na UD4
const usuario = ref(null)   // Ex: { id: 3, nome: 'María' }
const tarefas = ref([])

// Instanciamos a store de usuario
const usuarioStore = useUsuarioStore()

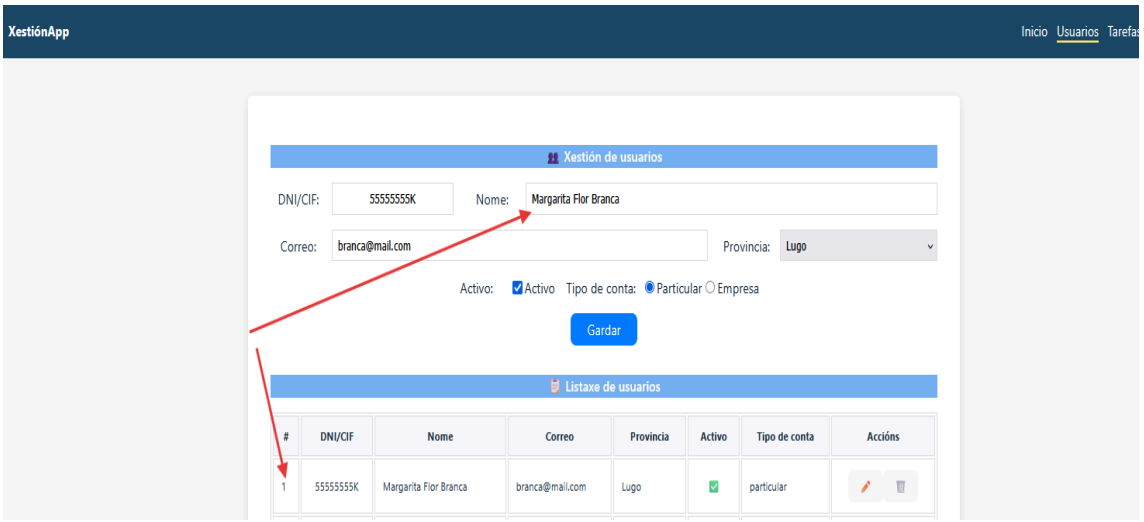
// Cargar Tarefas do usuario seleccionado
onMounted(() => {
  usuario.value = {
    id: usuarioStore.id,
    nome: usuarioStore.nome,
  }
```

Chegados a este punto xa podemos ver o funcionamento do ***Store***.

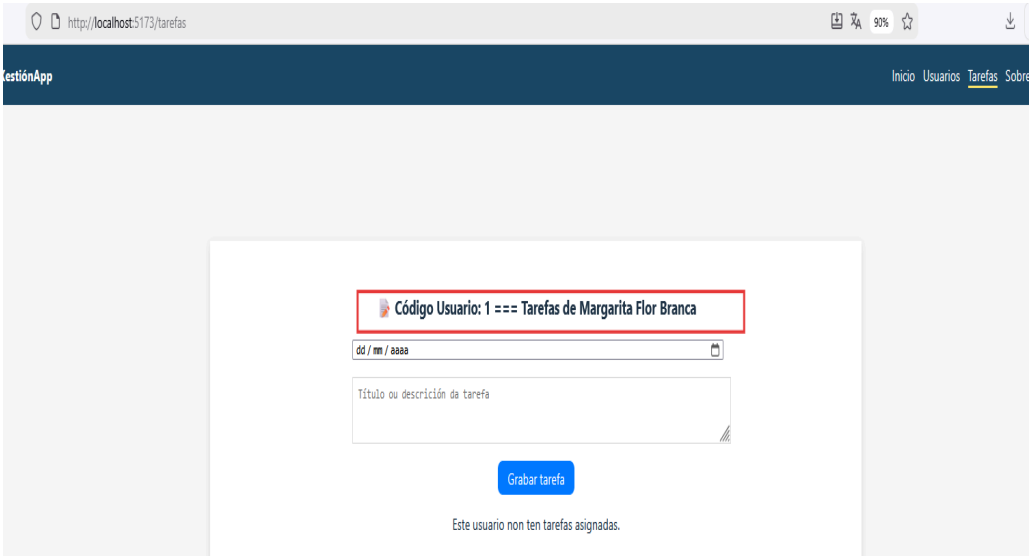
Pero antes, facemos unha pequeno cambio na zona do ***template*** en ***XestionTarefas.vue*** de maneira que cargue, de momento o *id* e o *nome* do usuario que seleccionamos.

```
src > components > XestionTarefas.vue > ...
1   <template>
2     <div class="xestion-tarefas">
3       <h3 v-if="usuarioStore.id">
4         Código Usuario: {{ usuarioStore.id }} === Tarefas de {{ usuarioStore.nome }}
5       </h3>
6
7       <p v-else>Non hai usuario seleccionado</p>
8
9       <!-- FORMULARIO NOVA TAREFA -->
10      <form @submit.prevent="engadirTarefa" class="form-tarefa">
11        <input v-model="novaTarefa.dataLimite" type="date" required />
```

Así, cando cargamos un usuario en ***XestionUsuarios.vue***



E vamos agora a ***XestionTarefas.vue***



Temos os datos do ***id*** e o ***nome*** cargados. Xa só nos queda gardar e xestionar as tarefas de cada usuario.

Só traballaremos as seguintes funcións:

- Listar as tarefas de cada usuario: ***cargarTarefas***
- Gardar novas tarefas para cada usuario: ***grabarTarefas***
- Modificar o estado das tarefas: ***completarTarefa***

Empecemos coas ***declaracións,*** despois con ***cargarTarefas*** en ***onMounted()*** para que cando se monte o compoñente cargue as tarefas do usuario, e ***onUnMounted()*** para limpar o *Store* cando salimos do compoñente.

Podemos ver as importacións, as declaración dende o ***Store*** e cando cargamos as tarefas de usuario cando se monta o compoñente e limpar o ***Store*** cando saimos do compoñente, algo que moitos programadores non fan e poden crear problemas cos datos.

```
<script setup>
import { ref, onMounted, onUnmounted } from "vue";
import axios from "axios";
import { useUsuarioStore } from "../store/usuarioStore.js";

const API_URL = import.meta.env.VITE_API_URL;
// Estes datos virán do store ou das peticións HTTP na UD4

const usuario = ref(null); // Ex: { id: 3, nome: 'María' }
const tarefas = ref([]);

// Instanciamos a store de usuario
const usuarioStore = useUsuarioStore();

// Nova tarefa
const novaTarefa = ref({
  titulo: "",
  dataLimite: "",
});


// Cargar Tarefas do usuario seleccionado
onMounted(async () => {
  // Aquí faríamos unha petición HTTP para obter as tarefas do usuario
  if (!usuarioStore.id) return;

  // esto podría facerse ou directamente desde a store ou ben con computed
  usuario.value = {
    id: usuarioStore.id,
    nome: usuarioStore.nome,
  };

  // Petición HTTP para obter as tarefas do usuario seleccionado
  try{
    const res = await axios.get(
      `${API_URL}/tarefas?usuarioId=${usuario.value.id}`
    );
    tarefas.value = res.data;
  } catch (error) {
    console.error("Erro ao cargar as tarefas:", error
    )
  }
});

// Limpar datos ao desmontar o compoñente
  onUnmounted(() => {
    usuarioStore.limparUsuario();
    tarefas.value = [];
  });
```

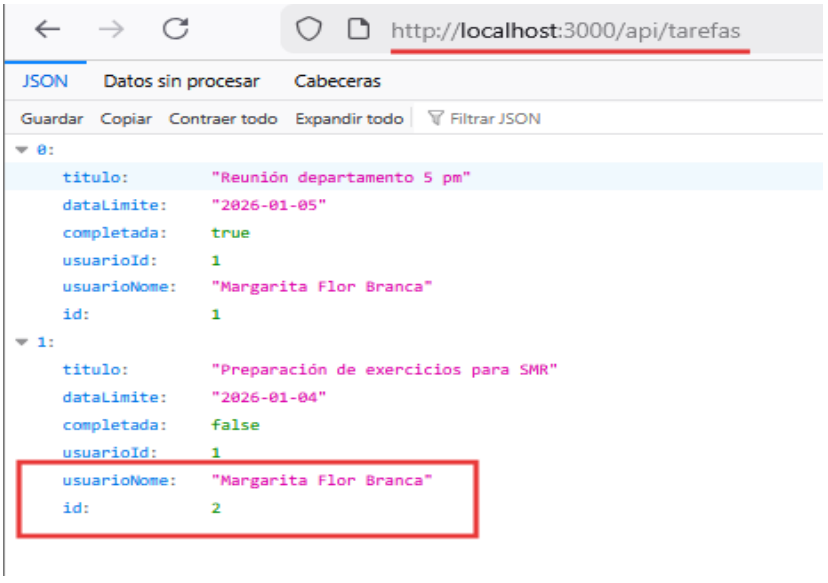
A segunda modificación e codificar o módulo ***cargarTarefas*** que carga as tarefas do usuario que temos no ***Store.*** Usamos ***POST,*** pero o importante e ver como asociamos a tarefa creada ao usuario pasado polo ***Store.***

```
// Crear tarefa (POST máis adiante con axios)
async function engadirTarefa() {
  try {
    // Construír obxecto nova tarefa
    const nova = {
      titulo: novaTarefa.value.titulo,
      dataLimite: novaTarefa.value.dataLimite,
      completada: false,
      usuarioId: usuarioStore.id,
      usuarioNome: usuarioStore.nome,
    };
    // Petición POST para gardar a nova tarefa
    const res = await axios.post(`${API_URL}/tarefas`, nova);
    // Engadir a tarefa á lista local e cargue a taboa
    tarefas.value.push(res.data);

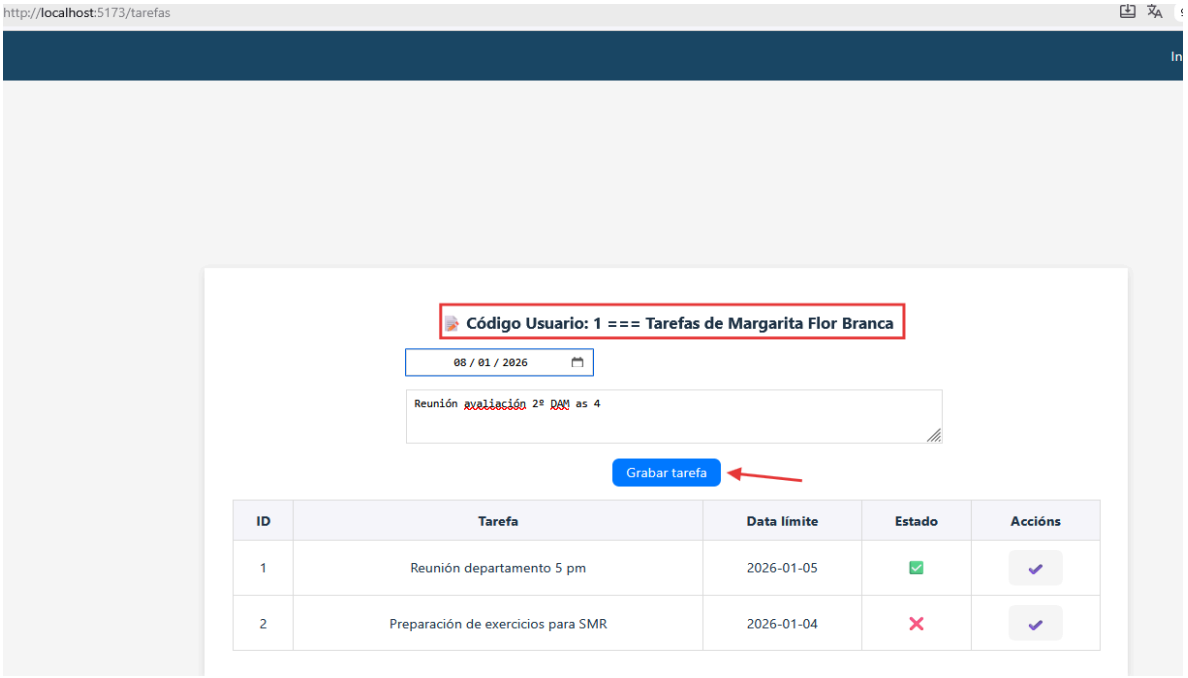
    novaTarefa.value.titulo = "";
    novaTarefa.value.dataLimite = "";
  } catch (error) {
    console.error("Erro ao gardar tarefa", error);
  }
}

// Marcar tarefa como completada (PUT máis adiante)
```

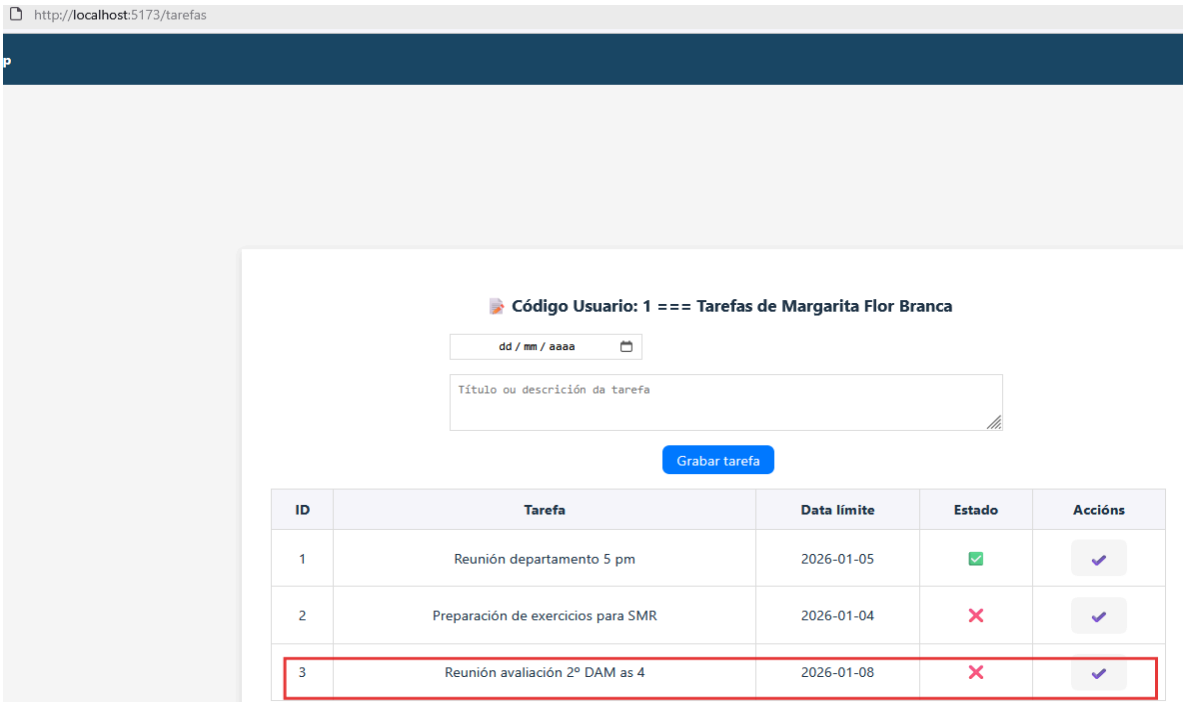
Se vemos no ***db.json***



E na táboa:



Tras engadir tarefa:



Finalmente, quédanos o código que tras pulsar o botón en ***Accións*** pasa o estado da tarefa de pendente a feito. Neste caso, tamén hai que botarlle un vistazo ao ***template*** para entender como a zona de ***script*** conecta coa zona ***template.***

```
<tbody>
  <tr v-for="(t, index) in tarefas" :key="index">
    <td style="text-align: center">{{ index + 1 }}</td>
    <td>{{ t.titulo }}</td>
    <td style="text-align: center">{{ t.dataLimite }}</td>
    <td style="text-align: center">
      {{ t.completada ? "✅" : "❌" }}
    </td>
    <td style="text-align: center">
      <button @click="completarTarefa(index)" :disabled="t.completada">
        ✔
      </button>
    </td>
```

Si *a t.completada* é ***True*** pon a etiqueta verde, pero si está a ***False*** entón pon a aspa vermella.

Ademais, algo **importante**, unha vez máis temos unha directiva ***v-bind*** ou "**:**" no botón, onde si *t.completada* está a ***True*** deshabilita o botón xa que non ten sentido. Podería ser que quixésemos cambiar o estado pero iso xa dependería dos requirimentos do usuario final.

Quédanos ver o código que manexa o ***template***:

```
// Marcar tarefa como completada (PUT máis adiante)
async function completarTarefa(index) {
  const tarefa = tarefas.value[index];

  try {
    const res = await axios.patch(
      `${API_URL}/tarefas/${tarefa.id}`,
      { completada: true }
    );

    tarefas.value[index] = res.data;
  } catch (error) {
    console.error("Erro ao actualizar tarefa", error);
  }
}
```

Como vemos, si a tarefa existe búscase e pon o campo **completada** a ***True.*** Cando se carga a táboa se o campo da tarefa está ***True*** ponse ca etiqueta verde como dixemos, ademais de deshabilitar o botón, e se encontra a ***False*** pon a aspa vermella e habilita o botón.

Ata aquí teríamos unha *app* sinxela pero completa xa funcional. Só nos quedaría configurar un servidor, compilar a aplicación con ***npm run build*** e subir todo ao servidor. Pero sería outra historia.

A.5. Xestión de estado baseada en parámetros de ruta

Ata agora estamos a usar ***Pinia*** para gardar o usuario seleccionado, o cal está perfecto para entender o concepto de estado global. Pero nesta sección imos cambiar o punto de vista: xa non é o usuario quen "vive" na aplicación, senón que é a **URL a que define o contexto da vista.**

De feito, nas aplicacións reais tamén é moi habitual que:

- A vista dependa da **URL**
- Se poida **recargar a páxina** sen perder o contexto
- Se poida **compartir unha URL** directamente (por exemplo, tarefas do usuario 3)

Para isto teríamos que:

1. No arquivo ***index.js*** establecer a seguinte ruta con **parámetro, onde *id*** sería o *id* do usuario.

```
const routes = [
  { path: '/', name: 'inicio', component: IniCio },
  { path: '/usuarios', name: 'usuarios', component: XestionUsuarios },
  { path: '/tarefas', name: 'tarefas', component: XestionTarefas },
  { path: '/sobrenos', name: 'sobrenos', component: SobreNos },
  { path: '/avisolegal', name: 'avisolegal', component: AvisoLegal },
  { path: "/tarefas/:id", name: "xestionTarefas", component: XestionTarefas }
```

2. No compoñente ***XestionUsuarios.vue*** faríamos a seguinte modificación, soamente na zona do ***template*** concretamente na zona da táboa onde engadimos a ruta

```
<td>{{ u.tipoCuenta }}</td>
<td style="text-align: center">
  <button @click="editarUsuario(index)" title="Editar">✏️</button>
  <button @click="eliminarUsuario(index)" title="Eliminar">🗑️</button>
  <router-link
    :to="{ name: 'xestionTarefas', params: { id: u.id } }"
    class="btn"
  >
    📄 Tarefas
  </router-link>
</td>
</tr>
```

A vista quedaría así:



3. No compoñente *XestiónTarefas.vue* e onde van o maior número de modificacións. Tampouco moitas.

   a. En primeiro lugar na zona de declaracións. cando accedemos á vista ***XestiónTarefas.vue*** a través dun enlace de navegación, entramos nun escenario distinto: **estamos cambiando de ruta**.

```
58  <script setup>
59  import { ref, onMounted, onUnmounted } from "vue";
60  import axios from "axios";
61  import { useUsuarioStore } from "../store/usuarioStore.js";
62  import { useRoute } from "vue-router";
63  import { watch } from "vue";
64
65  const route = useRoute();
66
67  const API_URL = import.meta.env.VITE_API_URL;
68  // Estes datos virán do store ou das peticións HTTP na UD4
```

   b. Por outro lado, **comentamos os códigos** de ***onMounted*** e de ***onUnMounted,*** que son substituídos por ***watch***

   Con ***watch*** empregamos esta función para **detectar cambios no parámetro *id*** da ruta. Isto permítenos **reaccionar automaticamente** cada vez que se selecciona un usuario distinto desde ***XestiónUsuarios.vue*** cando pulsamos **o botón tarefas.**

```
// Watch para cargar datos do usuario ao cambiar a ruta
watch(() => route.params.id,
  async (idUsuario) => {
    if (!idUsuario) return;

    try {
      // Cargar o usuario
      const resUsuario = await axios.get(
        `${API_URL}/usuarios/${idUsuario}`
      );

      usuario.value = {
        id: resUsuario.data.id,
        nome: resUsuario.data.nome,
      };

      // Cargar tarefas do usuario
      const resTarefas = await axios.get(
        `${API_URL}/tarefas?usuarioId=${idUsuario}`
      );
      tarefas.value = resTarefas.data;

    } catch (error) {
      console.error("Erro ao cargar datos", error);
    }
  },
  // Opcional: cargar inmediatamente ao montar o compoñente
  { immediate: true }
);
/*
// Cargar Tarefas do usuario seleccionado
onMounted(async () => {
  // Aquí faríamos unha petición HTTP para obter as tarefas do usuario
```

O que está "mirando" *watch* son os cambios en *params.id,* e dicir, dos cambios da *URL.*

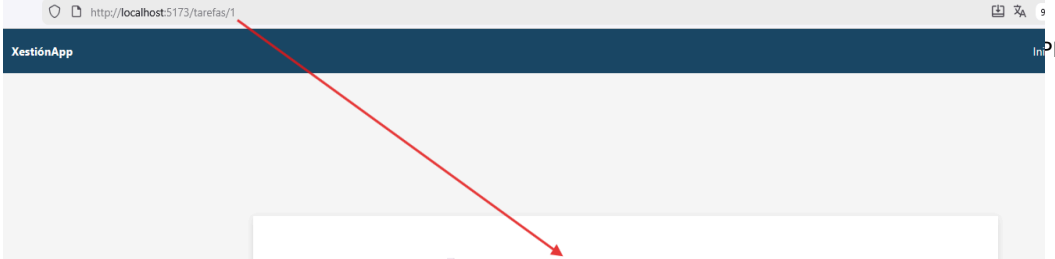
Quédanos unhas pequenas modificacións no ***template*** de ***XestionTarefas.vue*** que teñen que ver só co nome das variables para no confundilas coas de ***usuarioRoute***

```
<div class="xestion-tarefas">
  <!-- <h3 v-if="usuarioStore.id"> -->
  <h3 v-if="usuario">
  <!--   Código Usuario: {{ usuarioStore.id }} === Tarefas de {{ usuarioStore.nome }} -->
    Código Usuario: {{ usuario.id }} === Tarefas de {{ usuario.nome }}
  </h3>

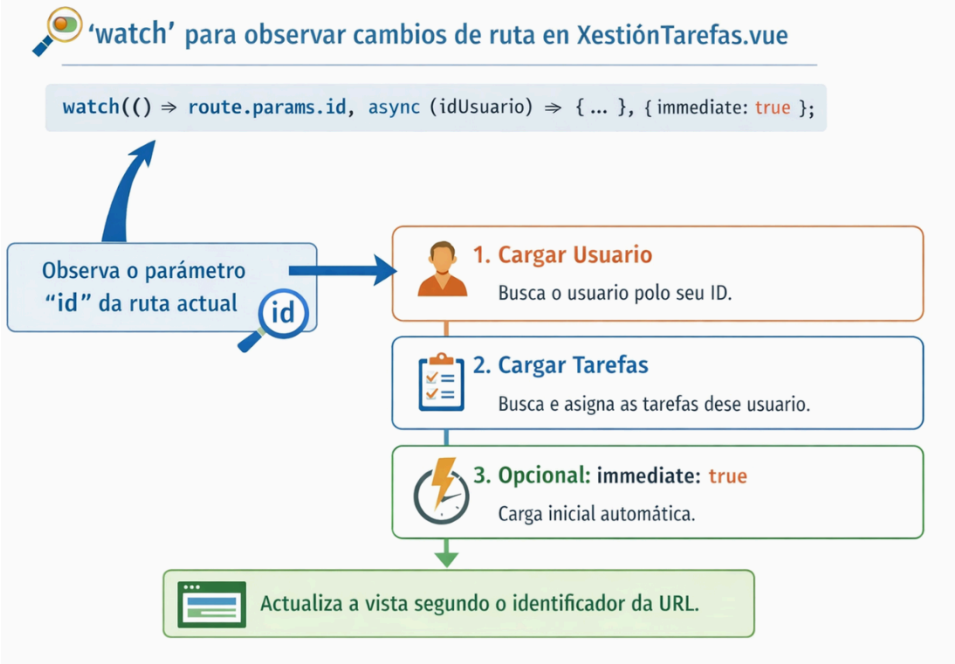
  <p v-else>Non hai usuario seleccionado</p>

  <!-- FORMULARIO NOVA TAREFA -->
  <form @submit.prevent="engadirTarefa" class="form-tarefa">
    <input v-model="novaTarefa.dataLimite" type="date" required />
```

Se nos fixamos na imaxe vemos que accedemos as tarefas de usuario polo ***id*** na *URL* ao pulsar o botón tarefas en ***XestionUsuarios.vue*** dende onde grazas a ***router-link*** carga o compoñentes ***XestionTarefas.vue*** con o parámetro ***id*** do usuario.



Resumindo, usamos ***watch*** para reaccionar aos cambios no parámetro da ruta. Cada vez que cambia o ***id*** do usuario na *URL*, *Vue* recarga automaticamente o usuario e as súas tarefas. A seguinte imaxe indica a sucesión de pasos que teñen lugar.



A.6. Que escoller nunha aplicación

En ***Vue 3*** existen distintas estratexias para compartir información entre compoñentes. Nesta unidade empregamos **dúas aproximacións complementarias**:

- o ***store global (Pinia)***
- e a **reacción aos cambios da URL mediante *watch***

Ambas son válidas, pero teñen **obxectivos distintos**.

Un ***store*** é un **almacén de estado global** accesible dende calquera compoñente da aplicación con as seguintes vantaxes:

- Código moi claro e centralizado
- Ideal para datos compartidos por moitos compoñentes
- Moi útil **para sesións, usuarios logueados, roles, preferencias**
- Evita repetir peticións *HTTP*

Pero adoece de:

- O estado **non é persistente** ao recargar a páxina
- Non queda reflectido na *URL*
- Non se pode compartir directamente cun ligazón

Con ***watch*** non gardamos o estado nun almacén global, senón que **delegamos esa responsabilidade na navegación,** sendo a *URL* é a *fonte da verdade* e o compoñente reacciona cando cambia o parámetro id.

Este método ten as súas vantaxes:

- A *URL* identifica completamente a vista
- Permite recargar a páxina sen perder contexto
- Posibilidade de copiar/compartir enlaces
- Patrón habitual en aplicacións profesionais

Cal escoller? Ambas dependendo do que se necesite.

En moitas aplicacións profesionais os parámetros da URL utilízanse para identificar o recurso que se está mostrando (por exemplo */usuarios/tarefas/2*). Aquí ***watch*** manda

O estado global, pola súa banda, utilízase para información que debe estar dispoñible en toda a aplicación, como **sesións de usuario** ou preferencias.

Concluíndo, nunha aplicación real non se escolle entre *Pinia* ou *watch*, senón que se empregan ambas ferramentas segundo o contexto e é como se traballa hoxe en día. ***Pinia*** utilízase para o **estado global** para ter datos dispoñibles en todo momento, mentres que os **parámetros da ruta e *watch*** permiten construír vistas dependentes da navegación, recargables e compartibles mediante *URL*. Esta combinación dá lugar a aplicacións máis claras, escalables e profesionais.

A.7. Liñas futuras de aprendizaxe

Con esta unidade remata a nosa iniciación en *Vue* 3, Pinia e *Router*. Agora contamos coas bases necesarias para construír aplicacións reactivas e escalables, e tedes unha visión clara das ferramentas que usaredes en proxectos máis complexos no futuro.

Aínda que o contido desta unidade pode resultar máis ou menos denso dependendo da experiencia previa de cada persoa, tomando esta base nun curso posterior poderían abordarse, entre outras, as seguintes liñas de traballo:

- **Autenticación e autorización:** calquera aplicación *web* profesional debe manexar aspectos como a xestión de usuarios, roles e xestión de sesións con *tokens*.
- ***Vue Router* avanzado:** uso de *navigation guards (beforeEach e beforeEnter), lazy loading* de vistas, estruturas anidadas, rutas públicas e privadas e xestión de error máis alá do 404.
- **Estado avanzado con Pinia:** con aspectos como *stores* modulares, persistencia de estes en *localStorage* e *sessionStorage* ou o uso combinado de *computed* dentro da store coa sincronización do *backend* e *store*.
- **Comunicación co *backend:*** configuración de *backend*s avanzados con *Mongodb* ou *MySql*, paxinación, filtros, interceptores de *Axios*.
- **Calidade e mantemento:** coa introdución ao *testing,* documentación, boas prácticas coas carpetas, organización profesional do proxecto ou tipado con *Typescript*.
- **Xeración do *build* do proxecto:** paso previo a subida e posta en produción.

A.8. Resumo

- **O emprego de Pinia** permítenos xestionar **estado global da aplicación**, compartindo datos entre compoñentes e vistas **sen usar *props* nin *emits*.**
- Os parámetros de ruta xunto con ***watch*** permiten crear **vistas reactivas á navegación**, recargables e dependentes da URL.
- Pinia e watch **non son excluíntes**: úsanse conxuntamente segundo se trate de estado da aplicación ou estado da navegación.